Sri Lanka Institute of Information Technology

DevOps Security (IE3142)

Lab Sheet 06

Year 3 Semester 1



Introduction to Dockers

Learning Objectives

- Explain why Docker is used and the difference between an image and a container.
- Pull images from Docker Hub and run containers.
- Use core Docker CLI commands to manage containers.
- Create a simple Dockerfile and build a custom image.
- Use Docker volumes for persistent data.
- Understand basic port mapping and Docker networking.
- Troubleshoot common beginner Docker errors and clean up resources.

Setup: Installing Docker on Kali Linux

Complete the following setup before beginning the practical exercises.

Step 1 - Update package lists

```
sudo apt update
```

Refreshes Kali's package information.

Step 2 - Install Docker

```
sudo apt install docker.io -y
```

Installs Docker from the Kali repositories.

Step 3 - Start Docker

```
sudo systemctl start docker
```

Starts the Docker service.

Step 4 - Enable Docker at boot

```
sudo systemctl enable docker
```

Configures Docker to start automatically when Kali boots.

Step 5 - Verify Docker

```
docker --version
sudo docker run hello-world
```

Checks the Docker version and runs the pre-built hello-world test image.

Part 0: Why Docker?

Docker helps solve the common "it works on my machine" problem by packaging an application with the runtime, libraries and configuration it needs. This improves consistency, isolation, portability and resource efficiency.

- Consistency - the same application environment can be used across development and deployment.
- Isolation - applications can run in separate containers.
- Speed - containers normally start much faster than full virtual machines.
- Efficiency - containers share the host operating-system kernel.
- Portability - the same image can be used on different Docker-enabled systems.

Part 1: Core Concepts

Term	Meaning
Image	A read-only template or blueprint used to create containers.
Container	An instance created from an image in which the application runs.
Dockerfile	A text file containing instructions used to build a Docker image.
Docker Hub	An online registry used to find and share Docker images.
Volume	Persistent storage managed separately from a container's lifecycle.
Docker Engine	The Docker service that builds and runs containers.

Containers vs. Virtual Machines

Feature	Virtual Machine	Container
Includes full OS?	Yes - each VM has its own guest OS/kernel	No - containers share the host kernel
Startup	Slower	Usually seconds
Size	Often gigabytes	Often much smaller
Resource usage	Heavier	Lighter
Isolation	Separate kernel	Process-level isolation

Part 2: Images and Running Containers

Exercise 2.1 - Pull an Image

```
sudo docker pull nginx
sudo docker images
```

The first command downloads the Nginx image. The second lists images stored locally.

Answer: _____

Exercise 2.2 - Run an Nginx Container

```
sudo docker run -d -p 8080:80 --name my-web nginx
```

- -d: runs the container in the background.
- -p 8080:80: maps host port 8080 to container port 80.
- --name my-web: names the container my-web.
- nginx: the image used to create the container.

Open a browser and visit http://localhost:8080. If port 8080 is already allocated, remove the failed container and use another available host port:

```
sudo docker rm my-web
sudo docker run -d -p 8081:80 --name my-web nginx
```

Then visit http://localhost:8081.

Exercise 2.3 - List Containers

```
sudo docker ps
sudo docker ps -a
```

docker ps shows running containers. docker ps -a shows all containers, including stopped containers.

Exercise 2.4 - Inspect a Container

```
sudo docker inspect my-web
```

Review the JSON output and identify the container's internal IP address.

Exercise 2.5 - Interactive Container

```
sudo docker run -it ubuntu bash
whoami
pwd
exit
```

The -it options provide an interactive terminal. Typing exit ends the Bash process and the container stops.

Part 3: Managing Containers

Exercise 3.1 - Logs

```
sudo docker logs my-web
sudo docker logs -f my-web
```

Use Ctrl+C to stop following live logs.

Exercise 3.2 - Execute Commands in a Running Container

```
sudo docker exec -it my-web bash
ls
cat /etc/os-release
cat /etc/nginx/nginx.conf
exit
```

docker run creates a new container; docker exec runs a command inside an existing running container.

Exercise 3.3 - Stop, Start and Restart

```
sudo docker stop my-web
sudo docker start my-web
sudo docker restart my-web
```

Exercise 3.4 - Resource Usage

```
sudo docker stats --no-stream
```

Answer: _____

Exercise 3.5 - Remove Containers

```
sudo docker stop my-web
sudo docker rm my-web
```

To force-remove a running container:

```
sudo docker rm -f my-web
```

Checkpoint Tasks

1. Run an Alpine container that prints "Hello Docker Lab" and exits.

```
sudo docker run alpine echo "Hello Docker Lab"
```

2. Run a BusyBox container named counter that prints the numbers 1 to 5, one per second.

```
sudo docker run --name counter busybox sh -c 'for i in 1 2 3 4 5; do echo $i; sleep
1; done'
```

Part 4: Building Your Own Image

Exercise 4.1 - Create the Application

Create a folder named docker-lab-app. Inside it, create app.py with:

```
print("Hello from my custom Docker image!")
```

Exercise 4.2 - Create the Dockerfile

Create a file named Dockerfile (with no file extension) in the same folder:

```
FROM python:3.12-slim
WORKDIR /app
COPY app.py .
CMD ["python", "app.py"]
```

Instruction	Purpose
FROM python:3.12-slim	Uses a lightweight Python base image.
WORKDIR /app	Sets /app as the working directory.
COPY app.py .	Copies app.py into the image.
CMD ["python", "app.py"]	Runs app.py when the container starts.

Exercise 4.3 - Build the Image

```
cd ~/Desktop/docker-lab-app
sudo docker build -t my-first-image .
```

The final dot (.) sets the current directory as the Docker build context.

Exercise 4.4 - Run the Custom Image

```
sudo docker run my-first-image
```

Expected output: Hello from my custom Docker image!

Exercise 4.5 - Modify and Rebuild

Change app.py to print your name, save it, rebuild the image, and run it again.

```
sudo docker build -t my-first-image .
sudo docker run my-first-image
```

Part 5: Volumes, Networking and Cleanup

Exercise 5.1 - Persist Data with a Volume

```
sudo docker run -d -p 8082:80 -v my-data:/usr/share/nginx/html --name web-with-volume
nginx
sudo docker volume ls
sudo docker volume inspect my-data
```

The volume my-data is mounted at /usr/share/nginx/html inside the Nginx container. A volume stores data independently of the container, allowing the data to persist when a container is replaced.

Exercise 5.2 - Docker Networking

```
sudo docker network ls
```

Review the default Docker networks. The default bridge network provides basic container networking.

Exercise 5.3 - Cleanup

Stop and remove the containers created during the lab:

```
sudo docker rm -f my-web web-with-volume counter
```

Remove the lab volume when it is no longer required:

```
sudo docker volume rm my-data
```

Optional cleanup command - use carefully because it removes unused Docker resources:

```
sudo docker system prune -a
```

Troubleshooting Reference

Problem	Likely Cause	Fix
Cannot connect to Docker daemon	Docker service is not running	sudo systemctl start docker
Permission denied	Insufficient Docker socket permissions	Use sudo or configure Docker group access
Port is already allocated	Host port is already in use	Use another port, e.g. -p 8081:80
Container name already in use	A container already has that name	sudo docker rm -f <container-name>
Container exits immediately	Main process finished or crashed	sudo docker logs <container-name>
Build file not found	Wrong folder/build context or filename	Check pwd, ls, Dockerfile and required files

Wrap-Up Questions

1. What problem does Docker help solve?

Answer: _____

2. What is the difference between a Docker image and a container?

Answer: _____

3. What is the purpose of a Dockerfile?

Answer: _____

4. How is a container different from a virtual machine?

Answer: _____

5. Why are Docker volumes used?

Answer: _____

6. What does port mapping such as 8080:80 mean?

Answer: _____

Quick Command Reference

Command	Purpose
docker pull <image>	Download an image
docker images	List local images
docker run <image>	Create and run a container
docker ps	List running containers
docker ps -a	List all containers
docker logs <name>	View container logs
docker exec -it <name> bash	Open Bash in a running container
docker stop/start/restart <name>	Manage container state
docker rm <name>	Remove a stopped container
docker build -t <name> .	Build an image from a Dockerfile
docker volume ls	List Docker volumes
docker network ls	List Docker networks